

Project - Student Database

Learning Objectives

- Design and implement a complete student database system using multiple data structures
- Apply CRUD operations: create, read, update, and delete student records
- Build search, reporting, and data export features with proper error handling

Engage (5-7 minutes)

A student database is a perfect project to apply everything learned in this chapter. It requires dictionaries for student records, lists for collections, tuples for immutable data like coordinates, and sets for unique tags. Today we build a real-world application that a school could actually use—combining data structures with functions, loops, and input handling into a complete system.

Explore (10-12 minutes)

Before writing code, design the database schema on paper. What information does each student record need? (Name, ID, age, class, subjects with marks, attendance.) How will you search for students? (By name, by ID, by class.) What reports do you need? (Class summary, individual report cards, honor roll.) Sketch the program menu and outline the main functions. Consider: how will you handle duplicate IDs? What about students who transfer classes?

Explain (10-12 minutes)

A CRUD system provides four core operations: Create (add new records), Read (view and search records), Update (modify existing records), and Delete (remove records). This project uses a dictionary as the main database with student IDs as keys. Each student record is a nested dictionary containing personal details and a dictionary of subject marks. The system uses functions for each operation, input validation to prevent errors, and formatted output for readability. Error handling ensures the system behaves gracefully when invalid data is entered.

Elaborate (8-10 minutes)

Build the system in phases: first, implement basic CRUD operations with a menu interface. Then add search functionality (by name, class, and ID range). Next, implement reporting features: class average, subject-wise analysis, honor roll (top 10%), and failing students. Add data persistence by saving to and loading from a text file. Finally, add input validation, error handling for edge cases, and a help menu. Test with 30 students across 3 classes and verify all calculations manually.

Evaluate (5-8 minutes)

Present your student database to a peer evaluator who will test it with unexpected inputs (empty names, negative marks, duplicate IDs, non-numeric ages). Document any bugs found and how you fixed them. Write a brief report covering: data structures used and why, functions implemented, edge cases handled, and one feature you would add for a production system.